

15_12_maggio

Allocazione dei frame

- **Paginazione su richiesta** (pure demand paging);
- quanti frame **assegnare** ad ogni processo?
 - **minimo:**
 - strutturale (set istruzioni, livelli di indirizzamento indiretto);
 - **massimo:** memoria libera;
- strategie:
 - allocazione **equa**;
 - allocazione **proporzionale**:
 - al processo i di dimensione s_i :
 - assegniamo ($a_i = s_i / S \times m$) frame;
 - dove, $S = \sum s_i$;
 - adeguamenti al livello di multiprogrammazione;
 - allocazione per **priorità**.

Allocazione dei Frame

Il problema è stabilire quanti frame assegnare a ciascun processo per bilanciare l'efficienza del sistema e minimizzare l'overhead causato dai page fault.

1. Paginazione su Richiesta (Pure Demand Paging) In questo scenario, il sistema non assegna preventivamente alcun frame al processo. All'inizio dell'esecuzione, tutte le pagine si trovano sul disco. È l'esecuzione stessa a determinare, tramite i page fault, quali pagine devono essere portate in RAM. Si osserva tipicamente un picco iniziale di page fault che tende poi a scendere verso una situazione di "normalità". Una frequenza troppo alta di page fault è problematica a causa dell'overhead di sistema che comporta.

2. Allocazione: Minimo e Massimo

• **Minimo Strutturale:** È il numero minimo di frame indispensabile perché un'istruzione possa essere eseguita. Dipende dall'architettura del set di istruzioni e dai livelli di indirizzamento indiretto. Ad esempio, se un'istruzione occupa più word (pagine diverse) o ha operandi multipli con indirizzamento indiretto, servono frame sufficienti a contenerli tutti contemporaneamente. • **Massimo:** Limite deciso dalla quantità totale di memoria libera nel sistema.

3. Strategie Non tutti i processi hanno le stesse esigenze. Le strategie principali includono:

- **Allocazione Equa:** Ogni processo riceve lo stesso numero di frame.
- **Allocazione Proporzionale:** I frame vengono assegnati in base alla "taglia" (spazio di indirizzamento) del processo.
 - $a_i = s_i / S \times m$, usiamo questa formula per assegnare i frame:
 - $a_i \rightarrow$ numero di frame
 - $s_i \rightarrow$ dimensione del processo (spazio di indirizzamento)
 - $S \rightarrow$ dimensione totale di tutti i processi nel sistema
 - $m \rightarrow$ numero totale di frame disponibili (RAM libera)
- **Allocazione per Priorità:** I frame vengono distribuiti in base all'importanza del processo piuttosto che alla sua sola dimensione.

Queste assegnazioni non sono statiche; devono adattarsi continuamente al livello di **multiprogrammazione**. Un principio fondamentale dell'allocazione riguarda la gestione del page fault: se un processo (es. P3) causa un fault, l'algoritmo di sostituzione dovrebbe idealmente selezionare una pagina da scartare tra quelle già assegnate a quel processo stesso, per evitare di sottrarre risorse ad altri processi, ma questa non è l'unica soluzione.

Allocazione dei frame

- Quali pagine considerare per la rimozione?
 - solo quelle dello stesso processo: **allocazione locale**;
 - tutte le pagine (anche di altri processi): **allocazione globale**;
- Cosa succede se ci sono **pochi frame** assegnati ad un processo?
 - sotto il **minimo strutturale**: viene sospeso e si fa swapping su disco;
 - un po' sopra:
 - può andare in **thrashing**;
 - quando il thrashing riguarda tutti i processi si parla di sistema in **sovraccarico** (eccessivo livello di multiprogrammazione);
- Bisognerebbe assegnare un numero di frame commisurato alle "**necessità** del processo":
 - **modello di località**;
 - concetto di **località**;
 - strategia di base.

1. Allocazione Locale e Globale

Allocazione Locale: La scelta della "vittima" da scartare ricade solo sulle pagine appartenenti allo stesso processo che ha causato il fault. Il numero di frame assegnati al processo rimane quindi costante.

Allocazione Globale: Il sistema può scegliere la vittima tra tutte le pagine presenti in memoria, incluse quelle di altri processi.

In questo caso, un processo può "rubare" un frame a un altro per favorire la propria esecuzione.

Qualunque algoritmo di sostituzione (FIFO, LRU, ecc.) può operare in entrambe le modalità; cambia solo l'insieme di pagine (il set) preso in considerazione.

Cosa succede se a un processo vengono assegnati troppi pochi frame?

Sotto il minimo strutturale: Il processo non ha abbastanza pagine per eseguire nemmeno una singola istruzione; viene quindi sospeso e spostato su disco (swapping).

Poco sopra il minimo (Thrashing): Si verifica il fenomeno del thrashing, una situazione in cui il sistema passa più tempo a scambiare pagine con il disco che a eseguire codice, a causa di un numero eccessivo di page fault.

Se il thrashing si propaga tra i processi a causa di un'allocazione globale gestita male, l'intero sistema va in sovraccarico. Questo accade spesso quando il livello di multiprogrammazione è troppo elevato per la RAM disponibile.

Il Concetto di Località

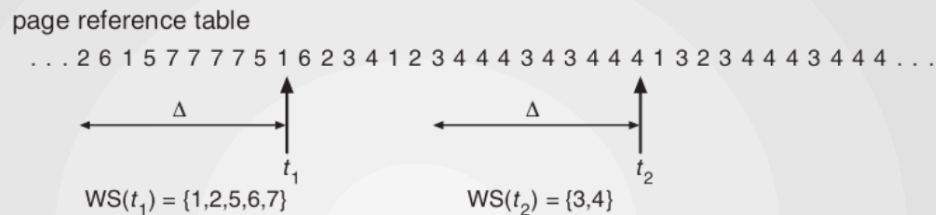
Per un sistema ideale, l'allocazione dovrebbe basarsi sulle reali esigenze del processo, concetto strettamente legato alla **località**:

- **Definizione di Località:** È l'insieme di informazioni (istruzioni e dati) necessarie all'esecuzione in una data fase di vita del processo.
- **Dinamicità:** La località muta nel tempo; ad esempio, il codice può passare dalla fase di inizializzazione a un ciclo di elaborazione dati differente.

Se il sistema riuscisse a identificare la dimensione della località attuale, potrebbe assegnare esattamente il numero di frame necessari per ospitarla. Garantire questo spazio permette al processo di lavorare correttamente senza thrashing.

Working set

- Per ogni processo manteniamo un **working set**:
 - pagine usate negli ultimi Δ accessi alla memoria;
 - scelta del **parametro** Δ per adattarlo alla **località corrente**;



- conoscendo il working set attuale di ogni processo:
 - richiesta globale di frame** ($D = \sum WSS_i$) vs. memoria disponibile;
 - prevenzione del **thrashing**
- come si calcola il working set in **pratica**? Si può approssimare usando:
 - interrupt** periodici;
 - bit di **referenziamento** R ;
 - un log che conserva la "storia di R " in base al parametro Δ .

© mario di raimondo

Working Set

Il **Working Set** è un concetto fondamentale per gestire la memoria basandosi sulla **località** di un processo. Si definisce come l'insieme delle pagine referenziate da un processo durante gli ultimi (Δ) accessi alla memoria.

- **Delta**: Rappresenta il numero di accessi, considerati per definire la località corrente.

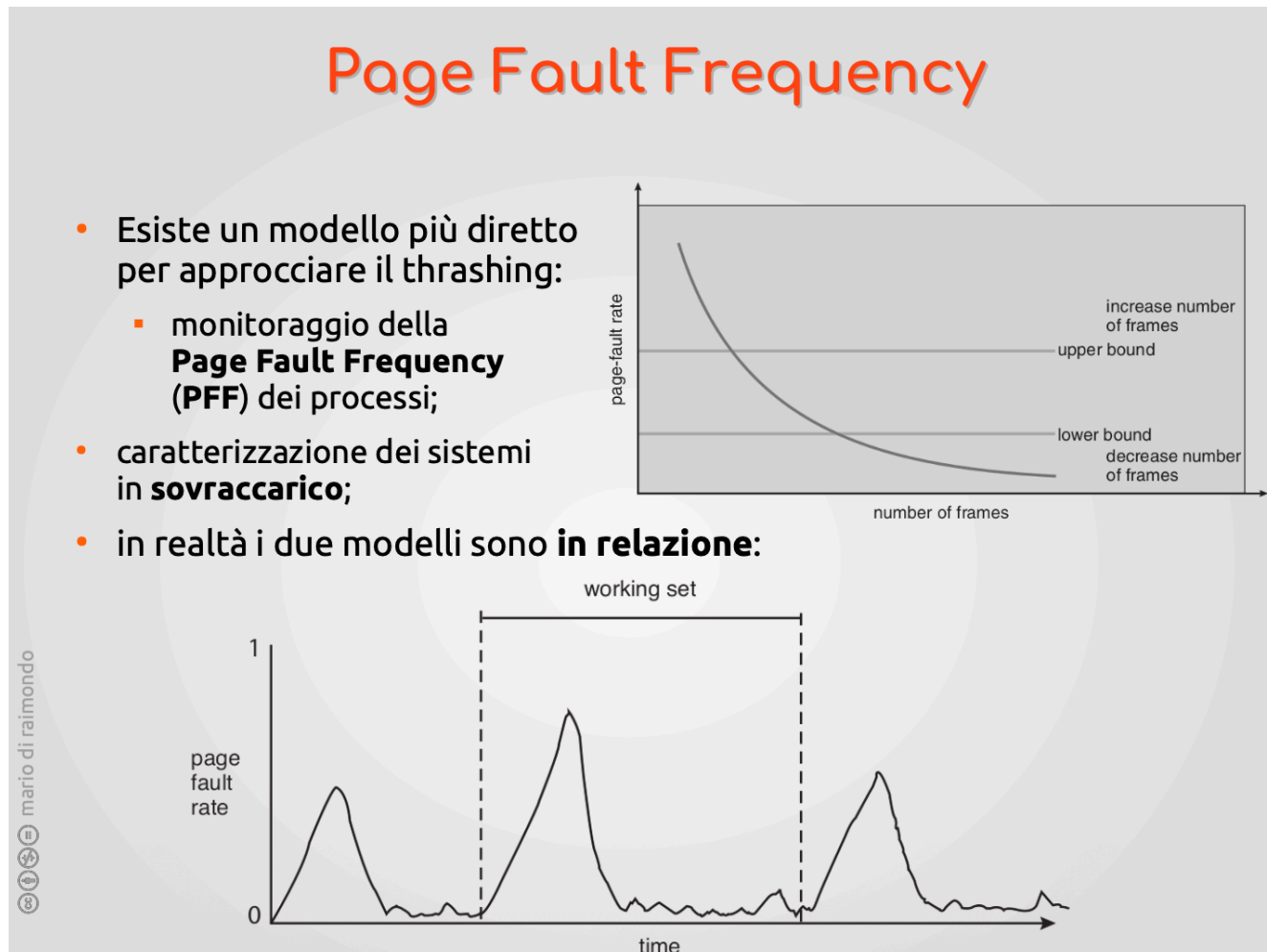
Anche all'interno dello stesso processo, il Working Set non è statico; la sua taglia e il suo contenuto cambiano nel tempo a seconda della fase di esecuzione.

Gestione del Thrashing Conoscere il Working Set permette al sistema operativo di monitorare la salute della memoria globale:

- **Richiesta Globale (D)**: Si calcola come la somma delle dimensioni dei working set di tutti i processi attivi.
- **Prevenzione del Thrashing**: Se la richiesta globale D supera la memoria fisica totale disponibile, il sistema entra in una situazione di sofferenza. Per prevenire il thrashing, il sistema può decidere di sospendere un processo (swapping) finché D non rientra nei limiti della capienza effettiva.

Tracciare il working set in maniera puntuale è complesso (simile a quanto accade per l'algoritmo LRU).

- **Interrupt periodici:** Il sistema controlla lo stato delle pagine a intervalli regolari.
- **Bit di referenziamento (R):** Si utilizza il bit hardware per capire se una pagina è stata toccata recentemente.
- **Log della "storia di R":** Viene conservato uno storico dell'utilizzo dei bit R per ricostruire quali pagine appartengono effettivamente alla finestra Delta stabilita.



Questo è un altro modello per prevenire il thrashing:

Mentre il **Working Set** cerca di prevedere di quali pagine un processo avrà bisogno, la **PFF** guarda cosa sta succedendo realmente "sul campo" misurando la frequenza dei page fault.

1. Il grafico in alto a destra: Controllo Dinamico

Il concetto fondamentale qui è l'uso di due soglie (limiti) per regolare il numero di frame assegnati a un processo:

- **Upper Bound (Soglia superiore):** Se la frequenza dei page fault supera questo limite, significa che il processo sta faticando troppo perché ha pochi frame a disposizione. Il sistema operativo reagisce **augmentando il numero di frame** assegnati a quel processo.
- **Lower Bound (Soglia inferiore):** Se la frequenza scende sotto questo limite, significa che il processo ha fin troppa memoria (più di quanta ne usi realmente). Il

sistema operativo può quindi **togliere frame** al processo per darli ad altri che ne hanno più bisogno.

- **Zona centrale:** È l'area di stabilità ottimale.

1. Il grafico in basso: Relazione con il Working Set

Questo grafico mostra come la frequenza dei page fault vari nel tempo in relazione alle fasi di esecuzione del programma:

- **I picchi:** Coincidono con l'inizio di una nuova fase del programma (un nuovo *Working Set*). Quando il processo cambia località (ad esempio passa da una funzione di calcolo a una di stampa), ha bisogno di nuove pagine che non sono in RAM, causando un'impennata di page fault.
- **Le valli:** Una volta che le pagine necessarie per la fase attuale sono state caricate in memoria (il *Working Set* è "stabile"), la frequenza dei page fault crolla quasi a zero.
- **Il rettangolo tratteggiato:** Indica il periodo in cui il processo sta caricando attivamente il suo working set attuale.

Politica di pulitura

- Il meccanismo di gestione dei page fault è efficiente soprattutto se ci sono **frame liberi** sempre disponibili;
- ciò velocizza la gestione dei page-fault;
- **paging daemon:** processo di servizio che controlla lo stato di occupazione globale dei frame del sistema;
 - seleziona, libera ed, eventualmente, pulisce pagine;
 - mantiene un **pool di frame liberi**;
 - possibilità di **ripescaggio** dal pool in caso di richiesta;
 - usato ad esempio su Linux e Windows.

Politica di pulitura

Quando una pagina viene portata in RAM, il sistema deve gestire non solo la sua presenza, ma anche il suo stato (come il **bit di modifica**) per ottimizzare le prestazioni.

Il meccanismo di gestione dei page fault è molto più efficiente se il sistema riesce a garantire la presenza di **frame liberi** pronti all'uso. Si aprono quindi due scenari:

1. Esiste almeno un frame libero. La richiesta di caricamento dal disco al frame può procedere immediatamente.
2. Tutti i frame sono occupati. In questo caso, il sistema deve prima liberare un frame (scelta della vittima e eventuale scrittura su disco), rallentando notevolmente l'operazione.

Per rimanere sempre nel "caso migliore", il sistema operativo cerca di mantenere una riserva o un **pool di frame liberi**.

Il Paging Daemon

Per garantire che ci siano sempre frame disponibili senza aspettare che la memoria si esaurisca, entra in gioco il **paging daemon**.

Un processo di servizio che controlla periodicamente lo stato di occupazione globale della memoria. Utilizza gli algoritmi di sostituzione per selezionare pagine "vittime" (scelte tra i processi che non le stanno utilizzando bene) per spostarle nel pool dei liberi. Se una pagina è stata modificata, il daemon si occupa di "pulirla" (scriverla su disco) prima di renderla disponibile nel pool.

Possibilità di **ripescaggio**: Se un processo richiede una pagina che è stata appena messa nel pool dei liberi ma non ancora sovrascritta, il sistema può recuperarla istantaneamente senza dover accedere al disco.

Dimensione della pagina

- La scelta della dimensione della pagina di base è importante:
 - vantaggi di una **pagina grande**:
 - ♦ tabella della pagine più piccola;
 - ♦ migliore efficienza nel trasferimento I/O;
 - ♦ tende a minimizzare il numero di page fault (minore overhead);
 - vantaggi di una **pagina piccola**;
 - ♦ minore frammentazione interna;
 - ♦ migliore risoluzione nel definire il working set in memoria (meno memoria sprecata);
- relazione con la **dimensione del blocco su disco**.

Dimensione della pagina

1. Vantaggi delle Pagine Grandi

L'utilizzo di pagine di dimensioni maggiori offre dei vantaggi:

- **Tabella delle pagine più grandi**: ogni pagina copre una porzione più ampia dello spazio di indirizzamento, sono necessarie meno voci nella tabella, riducendo l'occupazione di memoria della tabella stessa.
- **Efficienza nell'I/O**: Il trasferimento di dati tra disco e RAM è ottimizzato. È più veloce salvare o ripristinare pochi blocchi grandi rispetto a molti blocchi piccoli.
- **Riduzione dei Page Fault**: Tende a minimizzare il numero di fault, poiché ogni caricamento porta in memoria una quantità maggiore di dati correlati, riducendo l'overhead complessivo di gestione.

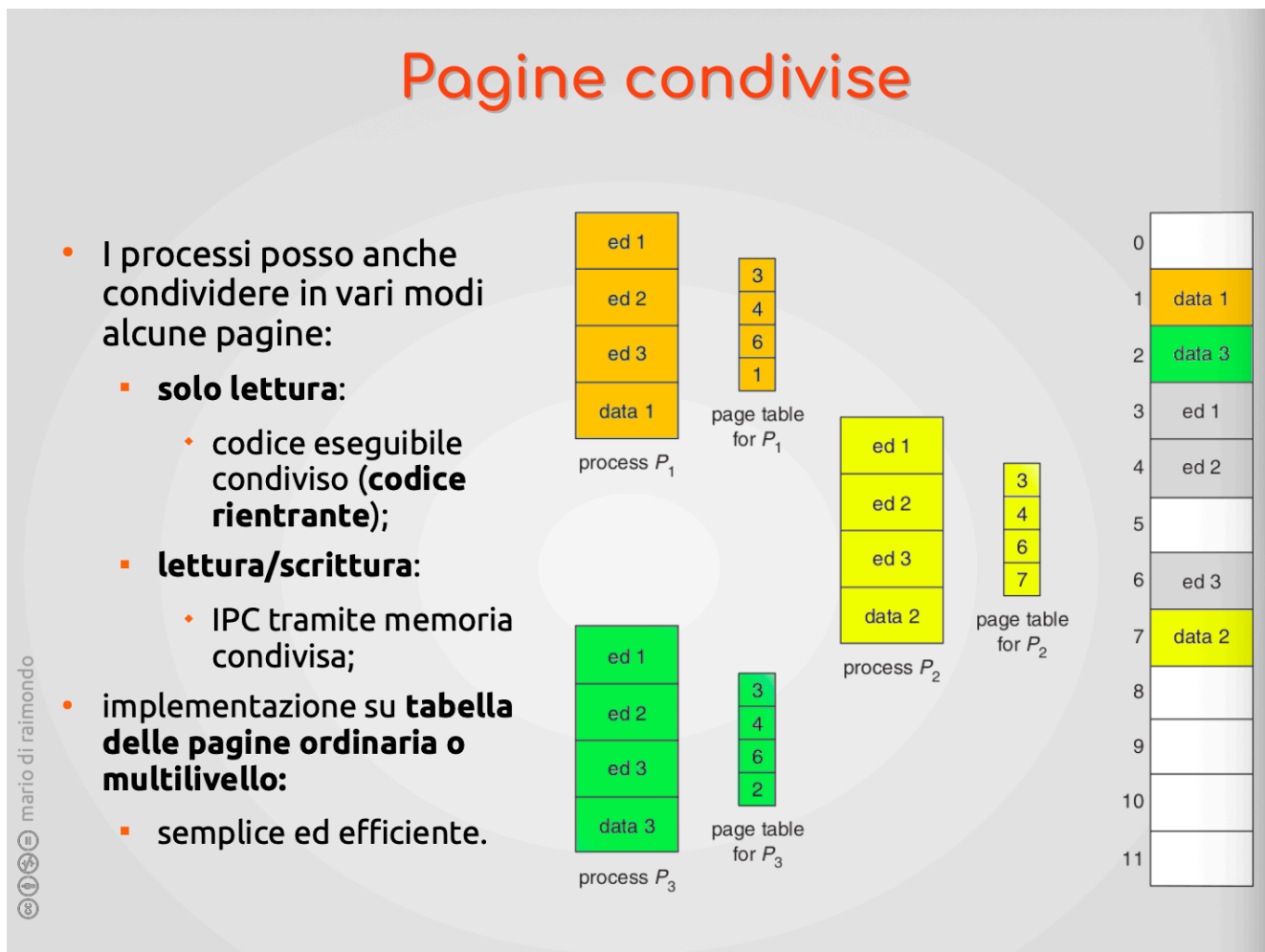
2. Vantaggi delle Pagine Piccole

Le pagine di dimensioni ridotte permettono una gestione più granulare e precisa:

- **Minore frammentazione interna**: Si riduce lo spazio sprecato all'interno dell'ultimo frame assegnato a un processo.
- **Migliore risoluzione del Working Set**: Permette di definire con maggiore precisione quali informazioni sono realmente necessarie in RAM. Con pagine grandi, si rischia

di occupare memoria con dati "inutili" che vengono consultati una sola volta ma che devono restare in RAM perché fanno parte di una pagina enorme. Questo si traduce in meno memoria sprecata.

Un fattore determinante nella scelta è la **dimensione del blocco su disco**. I dischi lavorano per blocchi di dimensione fissa; per ottimizzare i trasferimenti, la dimensione della pagina deve essere in correlazione con quella del blocco fisico.



Pagine condivise

Esistono situazioni in cui processi distinti, pur avendo spazi di indirizzamento separati e privati, hanno la necessità di condividere contenuti comuni. Questo avviene mappando diverse pagine virtuali di processi differenti sullo stesso frame di memoria fisica.

1. Condivisione in Sola Lettura (Codice Rientrante)

Questo scenario si verifica quando abbiamo istanze multiple dello stesso programma.

Il codice eseguibile può essere condiviso tra i processi P1, P2 e P3, le pagine virtuali ed1, ed2 ed3 di tutti e tre i processi puntano ai medesimi frame fisici (3, 4 e 6).

Ogni processo mantiene comunque il proprio stack e i propri dati privati (es. data 1, data 2), che risiedono in frame fisici separati.

2. Condivisione in Lettura/Scrittura

I processi possono chiedere esplicitamente al Sistema Operativo di mappare pezzi di memoria comuni per scambiarsi informazioni.

- **Memoria Condivisa:** Si crea una vera "finestra di comunicazione" tra i processi. Questo è un modello primordiale di comunicazione tra processi.
- In questo caso, i bit di protezione della tabella delle pagine abilitano anche la scrittura, permettendo ai processi di aggiornare i dati in tempo reale per coordinarsi.

Il meccanismo di implementazione è identico a quello della paginazione standard.

- Invece di caricare tre copie dello stesso codice in RAM, ne carichiamo solo una, liberando spazio per altre attività.

Con le pagine condivise è difficile gestire la cache

Pagine condivise

- Difficoltà:
 - **gestione della cache:**
 - ♦ **problemi di sincronizzazione** con cache basate su indirizzi virtuali (anche se usano gli ASID);
 - ♦ soluzioni:
 - ➔ disabilitare la cache sulle pagine condivise;
 - ➔ usare **cache con ricerca basata su indirizzi virtuali e tag fisici:**
 - la cache ricerca in parallelo con la TLB sulla base dell'indirizzo virtuale;
 - per capire se si tratta di un duplicato dobbiamo aspettare che la TLB dia in output l'indirizzo fisico;
 - **tabella delle pagine invertita:**
 - ♦ *singolo core*
 - ➔ alterazione tabella su context switch o su page fault;
 - ♦ *multi core*
 - ➔ difficilmente gestibile se non con teoriche tabelle delle pagine invertite con corrispondenze multi-a-uno.

La Tabella delle Pagine Invertita

A differenza delle tabelle standard (che hanno una voce per ogni pagina virtuale), esiste un'unica tabella per tutto il sistema che tiene traccia di cosa contiene ogni singolo frame fisico.

- La tabella contiene una voce per ogni frame della memoria RAM.

- Ogni voce indica quale processo (PID) e quale pagina virtuale specifica sono attualmente ospitati in quel frame.
- Quando la CPU genera un indirizzo virtuale, il sistema deve scansionare la tabella invertita per trovare la coppia (processo, pagina).
 - Se viene trovata una corrispondenza, si ottiene l'indice del frame e si accede alla memoria.
 - Se non c'è corrispondenza, si verifica un **page fault**.
- **Singolo Core:**
 - La tabella deve essere alterata o aggiornata a ogni **context switch** o al verificarsi di un **page fault**, aggiungendo overhead alla gestione del sistema.
- **Multi Core:**
 - La gestione diventa estremamente complessa (difficilmente gestibile).
 - Il problema principale sorge con le **pagine condivise**: poiché la tabella invertita nasce per avere una corrispondenza un frame = un processo, gestire corrispondenze un frame = più processi che lo condividono richiede soluzioni avanzate.